Platform Engineer Challenge — Open English

AI Quality Gate: a PR reviewer that raises the team's bar

Estimated time: 4–5 hours · **AI tools:** allowed and expected · **Delivery:** Git repository + review session (60 min)

Context

Open English is rebuilding its LMS from scratch with an AI-first team. The platform serves both adults and **minors** (Open English Junior), so children's privacy (GDPR-K, COPPA, and equivalent standards) is a design constraint, not a legal checklist.

As Platform Engineer, your job is to multiply the team's quality and speed. In this exercise you do it by building **an AI pipeline that reviews the team's work before it reaches production**.

Provided material

You receive a seed repository containing:

- A small LMS service (`student-progress`: tracks lesson progress; some students are minors).
- A `TEAM-STANDARDS.md` file with the team's rules (code quality, testing, PII handling, specific rules for minors' data).
- **8 open pull requests**, prepared by us. Some are sound. Others contain problems of varying severity and subtlety. You are not told which are which.

The exercise

Build an **AI Quality Gate**: a CI pipeline (GitHub Actions or similar) where an LLM reviews each PR against `TEAM-STANDARDS.md` and acts accordingly.

Part 1 — The reviewer

For each PR, the pipeline must:

1. **Review the diff with an LLM** against the team standards, with particular rigor on minors' privacy.

2. **Produce structured output**: findings with severity, location (file/line), rule violated, and suggested fix — as a PR comment or an equivalent report.
3. **Decide**: block the merge on severe violations (you define the threshold and justify it), comment on minor ones, approve the clean ones.
4. **Combine AI with deterministic checks where appropriate.** Not everything needs an LLM.

Part 2 — The eval harness

The 8 PRs are your golden set. Treat your reviewer as a system to be measured, not assumed. You must deliver:

1. **Ground truth**: your classification of the problems each PR contains (we reveal ours and compare during the review session).
2. **Reviewer metrics**: precision and recall of your pipeline against your ground truth.
3. **Failure analysis**: at least one false positive and one false negative analyzed — why they happened and what you would change.
4. **Trust policy**: which severities are blocked automatically vs. comment-only, justified with your metrics.

Out of scope (do not spend time here)

Real deployment of the service, UI, dashboards, fine-tuning. A pipeline running in CI (or locally against the PRs, well documented) is sufficient.

Ground rules

- **Use AI aggressively** to build (Claude, Cursor, Copilot). Deliver `AI-USAGE.md`: tools used, your 3–5 most important prompts, and at least one case where you discarded or corrected AI output and why.
- **Real 4–5 hour timebox.** Whatever does not fit, leave it prioritized in `DECISIONS.md` with the reasoning. Prioritizing well is worth more than covering everything.
- Everything must be reproducible from a README that a new developer can follow in **under 15 minutes**.

Deliverables

#	Deliverable	Description
1	Working pipeline	The quality gate running over the 8 PRs

#	Deliverable	Description
2	`EVAL.md`	Ground truth, metrics, failure analysis, trust policy
3	`README.md`	How to run everything in < 15 min
4	`DECISIONS.md`	3–6 mini-ADRs: trade-offs, what was left out and why
5	`AI-USAGE.md`	How you used AI to build (see above)

What we evaluate

Four dimensions, equally grounded in the deliverables above: the engineering quality of the AI pipeline, the rigor of your evaluation, the relevance of what your reviewer catches (with emphasis on minors' privacy), and the developer experience of the resulting tool. During the review session you will demo the pipeline, walk us through your decisions, and run it on a PR you have not seen.
